

Monitor de Ruido para Aulas

Memoria Técnica de Proyecto - SEyTR_Mo

Grupo: SEyTR_Mo Grupo 02

Autores: Enrique Padilla, Alexis Maestro, Sergio Muñoz, Alejandro Hernandez

Fecha: 05/05/2026

1. Introducción y Motivación

El ruido ambiental en las aulas dificulta la concentración y genera fatiga auditiva. Este proyecto implementa un sistema de monitorización acústica con Arduino para regular el volumen del aula mediante señales visuales y sonoras.

2. Materiales y Componentes

Componente	Función
Arduino Uno/Nano	Procesamiento y control de periféricos.
Sensor Sonido KY-037/38	Captación analógica (A2) de ondas sonoras.
LCD 16x2 I2C	Visualización de niveles y estados.
LEDs (V/A/R) + Buzzer	Alertas visuales y sonoras de estado.
Bluetooth HC-05	Comunicación inalámbrica con dispositivo móvil.

Coste Total: 45,84 €

3. Funcionamiento del Sistema

El sensor utiliza un micrófono **electret** y un comparador LM393. La señal analógica se procesa en el pin **A2** del Arduino siguiendo estos pasos:

1. Lectura del sensor: Realiza 250 lecturas rápidas para calcular la amplitud pico a pico (máximo – mínimo), medida relativa de la intensidad.

2. Filtrado de la señal: Almacena las últimas 5 mediciones en un historial y calcula una **media móvil** para estabilizar la respuesta.

- 3. Evaluación por umbrales:** Define niveles para ruido moderado o alto combinando el valor actual (picos) y la media móvil (continuado).
- 4. Visualización en LCD:** Muestra el nivel relativo y estados: "TODO EN ORDEN", "AVISO MODERADO" o "¡ALERTA ROJA!".
- 5. Alarma sonora:** Activa el zumbador con frecuencias distintas según la gravedad, controlando la duración para no viciar la lectura del sensor.
- 6. Envío por Bluetooth:** Transmite al móvil el ruido máximo, el estado y el modo (9600 baudios).
- 7. Modo manual:** Permite forzar estados (V, A, R) o volver a automático (M) vía app móvil.

4. Código Fuente Completo

```
#include
#include
#include

LiquidCrystal_I2C lcd(0x27, 16, 2);
SoftwareSerial bluetooth(4, 7);

const int pinBuzzer = 6;
const int pinVerde = 8;
const int pinAmarillo = 9;
const int pinRojo = 10;
const int pinSensor = A2;

const int UMBRAL_AMARILLO = 13;
const int UMBRAL_ROJO = 50;
const int UMBRAL_PICO_AMARILLO = 13;
const int UMBRAL_PICO_ROJO = 50;

int estadoActual = 0;
bool modoManual = false;

int historial[5] = {0, 0, 0, 0, 0};
int indice = 0;

unsigned long ultimoEnvio = 0;
unsigned long ultimoEstadoRojo = 0;
int ruidoMaxEnvio = 0;
const int INTERVALO_ENVIO = 200;
const int DURACION_ROJO = 1000;

int estadoBuzzer = 0;
unsigned long inicioBuzzer = 0;
unsigned long finBuzzer = 0;
const int DURACION_PITIDO_AMARILLO = 2000;
const int DURACION_PITIDO_ROJO = 3000;
const int BLOQUEO_SENSOR = 500;

int leerSensor() {
    int minimo = 1023;
    int maximo = 0;
    for (int i = 0; i < 250; i++) {
        int lectura = analogRead(pinSensor);
        if (lectura > maximo) maximo = lectura;
        if (lectura < minimo) minimo = lectura;
        delayMicroseconds(300);
    }
    return maximo - minimo;
}

void apagarLeds() {
    digitalWrite(pinVerde, LOW);
    digitalWrite(pinAmarillo, LOW);
    digitalWrite(pinRojo, LOW);
}

void leerBluetooth() {
    while (bluetooth.available()) {
        char comando = bluetooth.read();
        if (comando == '\n' || comando == '\r' || comando == ' ') continue;
        if (comando == 'V' || comando == 'v') {
            modoManual = true; estadoActual = 0;
            bluetooth.println(">> Modo manual: VERDE");
        } else if (comando == 'A' || comando == 'a') {
            modoManual = true; estadoActual = 1;
            bluetooth.println(">> Modo manual: AMARILLO");
        } else if (comando == 'R' || comando == 'r') {
            modoManual = true; estadoActual = 2;
            bluetooth.println(">> Modo manual: ROJO");
        } else if (comando == 'M' || comando == 'm') {
            modoManual = false; estadoActual = 0;
            bluetooth.println(">> Modo automatico activado");
        }
    }
}

void gestionarBuzzer() {
```

```

unsigned long ahora = millis();
if (estadoBuzzer == 1) {
    int duracion = (estadoActual == 2) ? DURACION_PITIDO_ROJO : DURACION_PITIDO_AMARILLO;
    if (ahora - inicioBuzzer > duracion) {
        noTone(pinBuzzer);
        finBuzzer = ahora;
        estadoBuzzer = 2;
    }
} else if (estadoBuzzer == 2) {
    if (ahora - finBuzzer > BLOQUEO_SENSOR) {
        estadoBuzzer = 0;
    }
}
}

void activarBuzzer(int frecuencia) {
    if (estadoBuzzer == 0) {
        tone(pinBuzzer, frecuencia);
        inicioBuzzer = millis();
        estadoBuzzer = 1;
    }
}

void setup() {
    pinMode(pinVerde, OUTPUT);
    pinMode(pinAmarillo, OUTPUT);
    pinMode(pinRojo, OUTPUT);
    pinMode(pinBuzzer, OUTPUT);
    Serial.begin(9600);
    bluetooth.begin(9600);
    Wire.begin();
    lcd.init();
    lcd.backlight();
    lcd.clear();
    lcd.print("SISTEMA ESTABLE");
    delay(1000);
}

void loop() {
    leerBluetooth();
    gestionarBuzzer();
    int ruido = (estadoBuzzer == 2) ? 0 : leerSensor();
    historial[indice] = ruido;
    indice = (indice + 1) % 5;
    int media = 0;
    for (int i = 0; i < 5; i++) media += historial[i];
    media = media / 5;
    if (!modoManual) {
        if (media > UMBRAL_ROJO || ruido > UMBRAL_PICO_ROJO) {
            estadoActual = 2;
            ultimoEstadoRojo = millis();
        } else if ((media > UMBRAL_AMARILLO || ruido > UMBRAL_PICO_AMARILLO) && estadoActual != 2) {
            estadoActual = 1;
        } else if (media < 10) {
            if (estadoActual == 2 && millis() - ultimoEstadoRojo > DURACION_ROJO) estadoActual = 0;
            else if (estadoActual != 2) estadoActual = 0;
        }
    }
    if (ruido > ruidoMaxEnvio) ruidoMaxEnvio = ruido;
    if (millis() - ultimoEnvio > INTERVALO_ENVIO) {
        ultimoEnvio = millis();
        String modo = modoManual ? "MANUAL" : "AUTO";
        String estado = estadoActual == 0 ? "VERDE" : estadoActual == 1 ? "AMARILLO" : "ROJO";
        bluetooth.print("Ruido: "); bluetooth.print(ruidoMaxEnvio);
        bluetooth.print(" | "); bluetooth.print(estado);
        bluetooth.print(" | "); bluetooth.println(modo);
        ruidoMaxEnvio = 0;
    }
    lcd.setCursor(0, 0);
    lcd.print(" Nivel:"); lcd.print(media); lcd.print(" ");
    apagarLeds();
    if (estadoActual == 2) {
        digitalWrite(pinRojo, HIGH);
        lcd.setCursor(0, 1); lcd.print("!ALERTA ROJA! ");
        activarBuzzer(2000);
    } else if (estadoActual == 1) {
        digitalWrite(pinAmarillo, HIGH);
        lcd.setCursor(0, 1); lcd.print("AVISO MODERADO ");
        activarBuzzer(500);
    } else {
        digitalWrite(pinVerde, HIGH);
        noTone(pinBuzzer);
        estadoBuzzer = 0;
        lcd.setCursor(0, 1); lcd.print("TODO EN ORDEN ");
    }
    delay(20);
}

```

```
}
```

5. Problemas y Soluciones

Problema: Baja sensibilidad del micrófono de serie a distancia.

Solución: Se incorporó un **cono acústico** físico que concentra las ondas sonoras hacia el micrófono, mejorando el rango de detección de forma mecánica y efectiva.

6. Conclusión

El proyecto es funcional y escalable. Se propone para el futuro el uso de sensores MAX4466 y placas ESP32 para dotar al sistema de conectividad IoT.